

THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY, PATIALA

Department of Electronics & Communication Engineering

PROJECT SEMESTER: 2025-26 Even Semester (from January 2026)

Internship Student GOALS REPORT (during First Visit)Date of Visit: 13th April, 2026

Roll No. & Name of Student: 102206125, Varnit Raj Singh

Name of Organization & Address: Excitel Broadband, Headquarters: Second & Third Floor, Plot No. 48, Okhla Industrial Estates, Phase-III, New Delhi, 110020

Phone No: 7626910250 Email: vsingh1_be22@thapar.edu

Name of Industry Mentor: Chirag Bhatnagar

Designation: NOC-L1 Phone No: 7977921321 Email: chirag.bhatnagar@dl.excitel.in

BRIEF GOAL REPORT**Topic/Title of Project:** Machine Learning Model Development and Deployment with Full-Stack Web Interface**Type of Project:** Software Development / Machine Learning / Full-Stack Web Application**Details of Project Semester Assignment:****1. Getting Started — Building the Foundation**

The first few weeks were spent getting comfortable with the core tools and technologies that would be used throughout the project. Since most of the work involves data and ML pipelines, it made sense to start from the ground up:

- Python: Started with the basics — syntax, loops, functions, and object-oriented concepts. Python was the obvious choice as the primary language since it's widely used in data science and ML.
- NumPy & Pandas: Learned how to work with numerical data using NumPy (arrays, matrix operations) and how to clean, explore, and manipulate datasets using Pandas. These two libraries form the backbone of almost any data project.
- Linux (Ubuntu Server): Got hands-on with the Linux terminal — navigating the file system, managing permissions, and running basic server operations. Working directly in a Ubuntu environment made it feel much more real-world.
- Networking Basics: Went through a guided playlist covering how switches, routers, IP addressing, and protocols work together. Understanding this helped later when setting up servers and containers that need to talk to each other.

2. Setting Up the Infrastructure

Once the basics were in place, the focus shifted to setting up a proper development environment and getting familiar with the tools professionals use day to day:

- Virtual Machines: Explored what VMs are, how hypervisors work, and spun up a virtual environment to experiment safely without affecting the host system.
- GitHub: Connected a GitHub repository and got comfortable with everyday version control — committing changes, pushing code, managing branches. Having a proper repo from day one made everything more organised.
- SSH Key Setup: Generated an SSH key pair and linked it to GitHub and the remote server. No more typing passwords — authentication is now secure and seamless.
- Cron Jobs: Learned how to schedule automated tasks in Linux using cron. As a hands-on exercise, a cron job was set up to run every 3 minutes and write its output to an output.log file — a simple but practical introduction to task automation.
- Docker: Spent considerable time understanding Docker from the ground up — what containers are, how images are built, writing Docker files, using docker-compose to manage multi-service setups, and thinking about how to deploy ML apps inside containers. This will be directly useful during the final deployment phase.

3. Machine Learning — Theory and Intuition

Before writing a single line of model code, time was invested in genuinely understanding how machine learning works — not just the formulas, but the intuition behind them. The study covered:

- Regression — understanding how models learn to predict continuous values, what loss functions are used, and why metrics like RMSE matter.
- Classification — both binary (yes/no decisions) and multi-class (picking between several categories), including how decision boundaries are formed.
- Image problems — a brief introduction to how models handle image data, to be explored further if time permits.
- LangChain: Got a high-level overview of LangChain, a framework for building applications on top of large language models. Covered the idea of chains, prompts, memory, and agents — interesting context for where the field is heading.

4. Building ML Models from Scratch

The core assignment is to build machine learning models entirely from scratch in Python — no shortcuts through pre-built libraries like scikit-learn for the model logic. The goal is to genuinely understand what is happening under the hood. Here is what will be built:

a) Models to Implement:

- For regression: Linear Regression and a Multi-Layer Perceptron (MLP) with manual backpropagation.
- For binary classification: Logistic Regression and an MLP.
- For multi-class classification: Softmax Regression and an MLP.

b) How the Project is Structured:

- Dataset Selection: Pick a real dataset, describe what it contains, decide whether the problem is regression or classification, and explain why a particular evaluation metric makes the most sense for it.
- Exploratory Data Analysis (EDA): Dig into the data before modelling — look at distributions, spot correlations, find outliers, and build up an understanding of what the data is actually telling us.
- Data Preprocessing: Split the data (70-30, 80-20, or 90-10), deal with missing values, encode any categorical columns, and scale the features so the model trains properly.
- Baseline Model: Write the model from scratch — initialise the weights, run the forward pass, compute the loss, calculate gradients, and update the weights using the rule: $w = w - lr * grad$. Every iteration's loss is stored so training progress can be tracked. A `predict()` function will return the model's output.
- Multi-Layer Perceptron (MLP): Build a deeper network with at least one hidden layer, ReLU activations, and an appropriate output activation. Implement forward propagation, backpropagation through the chain rule, and gradient descent — all manually coded. The architecture will follow the format: `[input, hidden1, hidden2, ..., output]`.

c) How Performance Will Be Measured:

- For regression models: MSE, RMSE, MAE, and R^2 to see how well the model predicts continuous values.
- For classification models: Accuracy, Precision, Recall, F1-Score, and a Confusion Matrix to understand where the model gets it right and where it struggles.

d) Visualisations:

- Training loss curves showing how the Baseline and MLP models improve over iterations — a clear way to see whether the model is actually learning.
- A bar chart comparing final performance across all models side by side.
- Any additional plots that are specific to the chosen dataset and help tell its story better.

5. Bringing It All Together — Deployment

The project does not end at model training. The final step is to make the models actually usable through a proper web interface — something a non-technical person could interact with:

- Backend API: A REST API built with Django or Flask will serve the trained models, accept input data and return predictions.
- Frontend: A React.js interface will let users interact with the model directly — enter values, hit a button, and see the prediction. Clean and functional.

6. Retrieval-Augmented Generation (RAG) Pipeline Implementation

As part of exploring modern LLM-based application architectures, a Proof of Concept (PoC) for a Retrieval-Augmented Generation (RAG) pipeline was developed using Python and the LangChain framework. The goal was to design a system capable of extracting, enriching, and structuring information from documents (PDFs) to enable efficient semantic retrieval.

a) Understanding and Initial Setup:

1. Studied the fundamentals of RAG architecture — combining retrieval systems with large language models to generate context-aware responses.

2. Explored LangChain components including document loaders, chains, embeddings, and vector stores.
3. Experimented with different APIs to understand integration patterns with LLMs.

b) PDF Parsing Pipeline:

A custom PDF processing pipeline was developed in Python to handle both textual and visual data within documents. The workflow is as follows:

1. PDF Upload:

The user uploads a PDF document into the system.

2. Text Extraction:

The textual content is extracted from the PDF and stored in a variable named `full_text_content`.

3. Image Processing and Enrichment:

- Images are extracted from the PDF.
- Optical Character Recognition (OCR) is applied using a free Python-based OCR tool to extract text from images.
- The extracted OCR text is appended to `full_text_content` with a label indicating "OCR Text".
- The same images are then passed to a Large Language Model (LLM), which generates concise and clear descriptions of each image.
- These descriptions are appended to `full_text_content` with the label "Image Description".

4. Final Output:

A unified and enriched textual representation of the PDF is generated, combining:

- Original text
- OCR-extracted text
- LLM-generated image descriptions

c) Chunking and Indexing:

1. The aggregated text (`full_text_content`) is processed using a recursive text chunking strategy to split the data into manageable and semantically meaningful chunks.
2. Each chunk is then converted into vector embeddings.
3. These embeddings are stored in a vector database (vector store) to enable efficient similarity-based retrieval.

d) Outcome:

This pipeline ensures that both textual and visual information from documents are captured and transformed into a searchable knowledge base. The approach significantly improves the quality of downstream retrieval and response generation in RAG-based applications.

(Signature of Faculty Mentor)

Name: _____

Designation: _____

(Signature of Industry Mentor)

Name: _____

Designation: _____